

Interface

Una interfaz en C# es como un contrato: define un conjunto de métodos y propiedades pero sin implementar su lógica (salvo excepciones predeterminadas).

Las clases que la utilizan están obligadas a programar esos comportamientos, lo que permite lograr polimorfismo y flexibilidad, ya que una clase puede implementar múltiples interfaces.

La declaración de una interfaz se realiza utilizando la palabra clave `interface`.

```
[modificador] interface NombreDeLaInterfaz
{
    // Definición de métodos
    tipoDeDato NombreDelMetodo(parametros);

    // Definición de propiedades
    tipoDeDato NombreDeLaPropiedad { get; set; }
}
```

- **NombreDeLaInterfaz** : Es el nombre único que se le da a la interfaz.
- **Modificador**: Puede ser `public` o `internal` para definir el nivel de acceso de la interfaz.
- **TipoDeDato**: Especifica el tipo de datos de las propiedades y métodos.
- **NombreDelMetodo, NombreDeLaPropiedad**: Son los identificadores únicos de los métodos, propiedades y eventos respectivamente.
- **Parametros**: Son las variables que se utilizan para pasar información al método cuando se llama.

A continuación, un ejemplo práctico de cómo se declara y utiliza una interfaz en un sistema de pagos:Ejemplo práctico:

Sistema de Pagos

Primero, definimos la interfaz **IPago**, que establece que cualquier método de pago debe saber cómo procesar un monto y validar su estado.

```
public interface IPago
{
    bool ProcesarPago(decimal monto);
    string ObtenerEstado();
}
```

Luego, diferentes clases implementan esta interfaz.

Cada una tendrá una lógica diferente, pero ambas cumplen exactamente con el mismo contrato.

```
public class PagoConTarjeta : IPago
{
    public bool ProcesarPago(decimal monto)
    {
        Console.WriteLine($"Procesando pago con tarjeta de ${monto}.");
    }
}
```

```

        return true;
    }

    public string ObtenerEstado() => "Pagado con tarjeta";
}

public class PagoConPayPal : IPago
{
    public bool ProcesarPago(decimal monto)
    {
        Console.WriteLine($"Procesando pago de ${monto} a través de PayPal.");
        return true;
    }

    public string ObtenerEstado() => "Pagado a través de PayPal";
}

```

¿Por qué utilizar interfaces?

- **Desacoplamiento:** Permite cambiar la implementación interna de una clase sin romper el código que la utiliza.
- **Simulación de herencia múltiple:** Como en C# una clase no puede heredar de varias clases padre, las interfaces permiten que una clase adquiera múltiples contratos y comportamientos diferentes.
- **Pruebas unitarias (Unit Testing):** Facilitan la creación de simulaciones (mocks) para probar componentes de manera aislada.

Otro ejemplo

```

public interface IConducible
{
    string Matricula { get; set; }

    // Métodos
    void Conducir();
    void Frenar();
}

public class Coche : IConducible
{
    // Implementación de propiedades
    public string Matricula { get; set; }

    // Implementación de métodos
    public void Conducir()
    {
        Console.WriteLine("El coche está en marcha.");
    }

    public void Frenar()
    {
        Console.WriteLine("El coche se ha detenido.");
    }
}

```

```
}  
}
```

Polimorfismo con interfaces

Una de las características más importantes de los interfaces es su capacidad para dar soporte al polimorfismo.

Esto permite que una instancia de una clase que implementa un interfaz sea tratada como una instancia de ese interfaz.

Supongamos que tenemos otra clase que implementa `IConducible` llamada `Bicicleta`.

```
public class Bicicleta : IConducible  
{  
    // implementación  
}
```

Ahora podemos definir una variable de tipo `IConducible`, y asignar variables de tipo `Coche` o `Bicicleta`.

```
IConducible vehiculo;  
  
vehiculo = new Coche();  
vehiculo.Arrancar(); // Salida: El coche ha arrancado.  
vehiculo.Detener(); // Salida: El coche se ha detenido.  
  
vehiculo = new Bicicleta();  
vehiculo.Arrancar(); // Salida: La bicicleta ha comenzado a moverse.  
vehiculo.Detener(); // Salida: La bicicleta se ha detenido.
```

Implementar múltiples interfaces

C# no soporta la herencia múltiple directa (una clase derivada de múltiples clases base). Sin embargo, una clase puede implementar múltiples interfaces, lo que proporciona una forma de lograr un comportamiento similar a la herencia múltiple.

Por ejemplo, en este ejemplo, la clase `Pato` implementa los interfaces `IVolador` e `INadador`.

```
public interface IVolador  
{  
    void Volar();  
}  
  
public interface INadador  
{  
    void Nadar();  
}  
  
public class Pato : IVolador, INadador  
{  
    public void Volar()  
    {
```

```

        Console.WriteLine("El pato está volando.");
    }

    public void Nadar()
    {
        Console.WriteLine("El pato está nadando.");
    }
}

```

Implementación explícita

C# permite la implementación explícita de miembros de interfaces. Esto es útil cuando una clase implementa múltiples interfaces que pueden tener métodos con el mismo nombre, o cuando se desea proporcionar implementaciones específicas que no sean accesibles directamente a través de la clase.

En este ejemplo, la clase Multifuncional implementa los interfaces IImprimible e IEscaneable. Pero ambos interfaces declaran un método Imprimir(). Podemos declarar explícitamente los interfaces para resolver el conflicto de nombres.

```

public interface IImprimible
{
    void Imprimir();
}

public interface IEscaneable
{
    void Imprimir();
}

public class Multifuncional : IImprimible, IEscaneable
{
    void IImprimible.Imprimir()
    {
        Console.WriteLine("Imprimiendo...");
    }

    void IEscaneable.Imprimir()
    {
        Console.WriteLine("Escaneando...");
    }
}

// Uso
IImprimible impresora = new Multifuncional();
impresora.Imprimir(); // Salida: Imprimiendo...

IEscaneable escaner = new Multifuncional();
escaner.Imprimir(); // Salida: Escaneando...

```

[Mas Info](#)

